

開始使用 AlloyDB 的混合型搜尋功能

來源：<https://codelabs.developers.google.com/next26/alloydb-ai-hybrid-search?hl=zh-tw>

步驟數：12

在本程式碼研究室中，您將瞭解如何使用 RUM 擴充功能和 (ScaNN) 索引，在 AlloyDB 中執行混合型搜尋，結合全文搜尋 (FTS) 和向量搜尋 (語意搜尋)。

目錄

1. [1. 簡介](#)
2. [2. 設定和需求](#)
3. [3. 事前準備](#)
4. [4. 部署 AlloyDB](#)
5. [5. 連線至 AlloyDB](#)
6. [6. 準備資料庫](#)
7. [7. 生成向量嵌入](#)
8. [8. 建立向量索引](#)
9. [9. 全文搜尋索引](#)
10. [10. 執行混合型搜尋](#)
11. [11. 清理環境](#)
12. [12. 恭喜](#)

步驟 1 · 約 0 分鐘

1. 簡介

在本程式碼研究室中，您將瞭解如何使用 (排名更新方法) RUM 擴充功能和可擴充最鄰近項目 (ScaNN) 索引，在 AlloyDB 中執行混合搜尋。本實驗室是 AlloyDB AI 功能專屬實驗室系列的一部分。詳情請參閱說明文件中的 [AlloyDB AI 頁面](#)。

必要條件

- 對 Google Cloud 控制台有基本瞭解
- 指令列介面和 Google Shell 的基本技能

課程內容

- 如何部署 [AlloyDB](#) 叢集和主要執行個體
- 如何從 [Google Compute Engine](#) VM 連線至 AlloyDB
- 如何建立資料庫並啟用 [AlloyDB AI](#)
- 如何將資料載入資料庫
- 如何使用 [AlloyDB Studio](#)
- 使用 [Vertex AI](#) 生成嵌入
- 如何建立 [ScaNN 向量索引](#)，提升向量搜尋效果
- 如何啟用及使用 RUM 擴充功能進行全文搜尋
- 結合全文搜尋、向量搜尋和[倒數排名融合 \(RRF\)](#) 執行混合型搜尋

軟硬體需求

- Google Cloud 帳戶和 Google Cloud 專案
 - 網路瀏覽器，例如 [Chrome](#)
-

步驟 2 · 約 5 分鐘

2. 設定和需求

info

從做中學 免付費

立即取得 Google Cloud 抵免額，並完成這個實驗室。無須提供信用卡或付款方式。

啟用

專案設定

登入 [Google Cloud 控制台](#)。如果沒有 Gmail 或 Google Workspace 帳戶，請先[建立帳戶](#)。

請改用個人帳戶，而非公司或學校帳戶。

注意：公司和學校帳戶可能設有限制，導致您無法啟用本實驗室所需的 API，或存取部分資源。

建立 Google Cloud 專案

1. 在 [Google Cloud 控制台](#) 的專案選取器頁面中，[選取或建立 Google Cloud 專案](#)。
2. 確認 Cloud 專案已啟用計費功能。瞭解如何[檢查專案是否已啟用計費功能](#)。

注意：專案 ID 在全域中是唯一的，選定後就無法再供他人使用。您是該 ID 的唯一使用者。即使專案已刪除，ID 也無法再次使用。

注意：如果使用 Gmail 帳戶，可以將預設位置設為「沒有機構」。如果你使用 Google Workspace 帳戶，請選擇適合貴機構的位置。

啟用計費功能

如要啟用計費功能，有兩種方法。你可以使用個人帳單帳戶，也可以按照下列步驟兌換抵免額。

設定個人帳單帳戶

如果使用 Google Cloud 抵免額設定計費，則可略過此步驟。

如要設定個人帳單帳戶，請[前往這裡在 Cloud 控制台中啟用帳單功能](#)。

注意事項：

- 完成本實驗室的 Cloud 資源費用應低於 \$3 美元。
- 您可以按照本實驗室結尾的步驟刪除資源，以免產生後續費用。
- 新使用者可獲得價值 [\\$300 美元的免費試用期](#)。

啟動 Cloud Shell

雖然可以透過筆電遠端操作 Google Cloud，但在本程式碼研究室中，您將使用 [Google Cloud Shell](#)，這是可在雲端執行的指令列環境。

Cloud Shell 是在 Google Cloud 中運作的指令列環境，已預先載入必要工具。

1. 點選 Google Cloud 控制台頂端的「啟用 Cloud Shell」。
2. 連至 Cloud Shell 後，請驗證您的驗證：

```
gcloud auth list
```

3. 確認專案已設定完成：

```
gcloud config get project
```

4. 如果專案未如預期設定，請設定專案：

```
export PROJECT_ID=<YOUR_PROJECT_ID>  
gcloud config set project $PROJECT_ID
```

這部虛擬機器搭載各種您需要的開發工具，並提供永久的 5GB 主目錄，而且可在 Google Cloud 運作，大幅提升網路效能並強化驗證功能。您可以在瀏覽器中完成本程式碼研究室的所有作業。您不需要安裝任何軟體。

步驟 3 · 約 5 分鐘

3. 事前準備

啟用 API

輸出內容：

請注意，啟用部分資源會產生費用。在正常情況下，如果實驗室完成後 AlloyDB 叢集和運算 VM 都遭到毀損，所有資源的費用不會超過 \$3 美元。建議您查看帳單，確認這項措施是否可接受。

如要使用 [AlloyDB](#)、[Compute Engine](#)、[網路服務](#)和 [Vertex AI](#)，您需要在 Google Cloud 雲端專案中啟用各自的 API。

啟用 API

在終端機的 Cloud Shell 中，確認專案 ID 已設定完畢：

```
gcloud config set project [YOUR-PROJECT-ID]
```

設定環境變數 PROJECT_ID：

```
PROJECT_ID=$(gcloud config get-value project)
```

啟用所有必要的 API：

```
gcloud services enable alloydb.googleapis.com \  
compute.googleapis.com \  
cloudresourcemanager.googleapis.com \  
servicenetworking.googleapis.com \  
aiplatform.googleapis.com
```

預期的輸出內容：

```
student@cloudshell:~ (test-project-001-402417)$ gcloud config set project test-
project-001-402417
Updated property [core/project].
student@cloudshell:~ (test-project-001-402417)$ PROJECT_ID=$(gcloud config get-value
project)
Your active configuration is: [cloudshell-14650]
student@cloudshell:~ (test-project-001-402417)$
student@cloudshell:~ (test-project-001-402417)$ gcloud services enable
alloydb.googleapis.com \
compute.googleapis.com \
cloudresourcemanager.googleapis.com \
servicenetworking.googleapis.com \
aiplatform.googleapis.com
Operation "operations/acat.p2-4470404856-1f44ebd8-894e-4356-bea7-b84165a57442"
finished successfully.
```

API 簡介

- **AlloyDB API** (`alloydb.googleapis.com`) 可讓您建立、管理及擴充 AlloyDB for PostgreSQL 叢集。這項服務提供與 PostgreSQL 相容的全托管資料庫服務，專為要求嚴苛的企業交易和分析工作負載而設計。
- **Compute Engine API** (`compute.googleapis.com`) 可讓您建立及管理虛擬機器 (VM)、永久磁碟和網路設定。這個服務提供執行工作負載所需的基礎架構即服務 (IaaS) 基礎，並代管許多代管服務的基礎架構。
- **Cloud Resource Manager API** (`cloudresourcemanager.googleapis.com`) 可讓您以程式輔助方式管理 Google Cloud 雲端專案的中繼資料和設定。您可以在專案階層中整理資源、處理 Identity and Access Management (IAM) 政策，以及驗證權限。
- **Service Networking API** (`servicenetworking.googleapis.com`) 可讓您自動設定虛擬私有雲 (VPC) 網路與 Google 代管服務之間的私人連線。您必須設定私人服務存取權，才能為 AlloyDB 等服務建立私人 IP 存取權，確保這些服務能與其他資源安全地通訊。
- **Vertex AI API** (`aiplatform.googleapis.com`) 可讓應用程式建構、部署及擴充機器學習模型。這個平台提供統一的介面，方便您使用所有 Google Cloud AI 服務，包括存取生成式 AI 模型 (如 Gemini) 和模型訓練。

您也可以選擇設定預設區域，使用 Vertex AI 嵌入模型。進一步瞭解 [Vertex AI 服務地區](#)。在本範例中，我們使用的是 us-central1 區域。

```
gcloud config set compute/region us-central1
```

4. 部署 AlloyDB

建立 AlloyDB 叢集前，我們需要虛擬私有雲中可用的私人 IP 範圍，供日後的 AlloyDB 執行個體使用。如果沒有，我們就必須建立，並指派給內部 Google 服務使用，之後才能建立叢集和執行個體。

建立私人 IP 範圍

注意：只有在您尚未指派未使用的私人 IP 範圍，與 Google 內部服務搭配使用時，才需要執行這個步驟。

我們需要在虛擬私有雲中為 AlloyDB 設定[私人服務存取權](#)。這裡的假設是專案中具有「預設」虛擬私有雲網路，且所有動作都會使用該網路。

建立私人 IP 範圍：

```
gcloud compute addresses create psa-range \
  --global \
  --purpose=VPC_PEERING \
  --prefix-length=24 \
  --description="VPC private service access" \
  --network=default
```

使用分配的 IP 範圍建立私人連線：

```
gcloud services vpc-peerings connect \
  --service=servicenetworking.googleapis.com \
  --ranges=psa-range \
  --network=default
```

注意：第二個指令需要幾分鐘才能執行完畢

預期的控制台輸出內容：


```
student@cloudshell:~ (test-project-402417)$ gcloud compute addresses create psa-range \
--global \
--purpose=VPC_PEERING \
--prefix-length=24 \
--description="VPC private service access" \
--network=default
Created [https://www.googleapis.com/compute/v1/projects/test-project-402417/global/addresses/psa-range].

student@cloudshell:~ (test-project-402417)$ gcloud services vpc-peerings connect \
--service=servicenetworking.googleapis.com \
--ranges=psa-range \
--network=default
Operation "operations/pssn.p24-4470404856-595e209f-19b7-4669-8a71-cbd45de8ba66"
finished successfully.

student@cloudshell:~ (test-project-402417)$
```

建立 AlloyDB 叢集

在本節中，我們將在 us-central1 區域建立 AlloyDB 叢集。

定義 postgres 使用者的密碼。您可以自行定義密碼，也可以使用隨機函式產生密碼

```
export PGPASSWORD=`openssl rand -hex 12`
```

預期的控制台輸出內容：

```
student@cloudshell:~ (test-project-402417)$ export PGPASSWORD=`openssl rand -hex 12`
```

請記下 PostgreSQL 密碼，以供日後使用。

```
echo $PGPASSWORD
```

日後以 postgres 使用者身分連線至執行個體時，需要使用該密碼。建議您記下或複製這組代碼，以供日後使用。

預期的控制台輸出內容：

```
student@cloudshell:~ (test-project-402417)$ echo $PGPASSWORD
bbefbfde7601985b0dee5723
```

建立 AlloyDB 叢集

定義區域和 AlloyDB 叢集名稱。我們將使用 us-central1 區域，並以 alloydb-hybrid-search 做為叢集名稱：

```
export REGION=us-central1
export ADBCLUSTER=alloydb-hybrid-search
```

注意：如果您尚未開始使用 AlloyDB，可以執行下列叢集建立指令，並加上 `--subscription-type=TRIAL` 旗標，建立[免費試用叢集](#)：

執行指令來建立叢集：

```
gcloud alloydb clusters create $ADBCLUSTER \
  --password=$PGPASSWORD \
  --network=default \
  --region=$REGION
```

預期的控制台輸出內容：

```
export REGION=us-central1
export ADBCLUSTER=alloydb-hybrid-search
gcloud alloydb clusters create $ADBCLUSTER \
  --password=$PGPASSWORD \
  --network=default \
  --region=$REGION
Operation ID: operation-1697655441138-6080235852277-9e7f04f5-2012fce4
Creating cluster...done.
```

在同一個 Cloud Shell 工作階段中，為叢集建立 AlloyDB 主要執行個體。如果連線中斷，您需要再次定義區域和叢集名稱環境變數。

注意：建立執行個體通常需要 6 到 10 分鐘

```
gcloud alloydb instances create $ADBCLUSTER-pr \
  --instance-type=PRIMARY \
  --cpu-count=2 \
  --region=$REGION \
  --cluster=$ADBCLUSTER
```

預期的控制台輸出內容：

```
student@cloudshell:~ (alloydb-hybrid-search)$ gcloud alloydb instances create
$ADBCLUSTER-pr \
  --instance-type=PRIMARY \
  --cpu-count=2 \
  --region=$REGION \
  --availability-type ZONAL \
  --cluster=$ADBCLUSTER
Operation ID: operation-1697659203545-6080315c6e8ee-391805db-25852721
Creating instance...done.
```

步驟 5 · 約 10 分鐘

5. 連線至 AlloyDB

AlloyDB 是透過僅限私人的連線部署，因此我們需要安裝 PostgreSQL 用戶端的虛擬機器，才能使用資料庫。

部署 GCE VM

在與 AlloyDB 叢集相同的區域和 VPC 中建立 GCE VM。

在 Cloud Shell 中執行下列指令：

```
export ZONE=us-central1-a
gcloud compute instances create instance-1 \
  --zone=$ZONE \
  --create-disk=auto-delete=yes,boot=yes,image=projects/debian-cloud/global/images/$(gcloud
compute images list --filter="family=debian-12 AND family!=debian-12-arm64" --
format="value(name)") \
  --scopes=https://www.googleapis.com/auth/cloud-platform
```

預期的控制台輸出內容：

```
student@cloudshell:~ (alloydb-hybrid-search)$ export ZONE=us-central1-a
student@cloudshell:~ (talloydb-hybrid-search)$ export ZONE=us-central1-a
gcloud compute instances create instance-1 \
  --zone=$ZONE \
  --create-disk=auto-delete=yes,boot=yes,image=projects/debian-
cloud/global/images/$(gcloud compute images list --filter="family=debian-12 AND
family!=debian-12-arm64" --format="value(name)") \
  --scopes=https://www.googleapis.com/auth/cloud-platform

Created [https://www.googleapis.com/compute/v1/projects/test-project-402417/zones/us-
central1-a/instances/instance-1].
NAME: instance-1
ZONE: us-central1-a
MACHINE_TYPE: n1-standard-1
PREEMPTIBLE:
INTERNAL_IP: 10.128.0.2
EXTERNAL_IP: 34.71.192.233
STATUS: RUNNING
```

安裝 Postgres 用戶端

在已部署的 VM 上安裝 PostgreSQL 用戶端軟體

連線至 VM：

注意：第一次建立 VM 的 SSH 連線時，可能需要較長時間，因為這個程序包括建立 RSA 金鑰以確保安全連線，以及將金鑰的公開部分傳播至專案

```
gcloud compute ssh instance-1 --zone=us-central1-a
```

預期的控制台輸出內容：

```
student@cloudshell:~ (alloydb-hybrid-search)$ gcloud compute ssh instance-1 --
zone=us-central1-a
Updating project ssh metadata...working..Updated
[https://www.googleapis.com/compute/v1/projects/alloydb-hybrid-search].
Updating project ssh metadata...done.
Waiting for SSH key to propagate.
Warning: Permanently added 'compute.5110295539541121102' (ECDSA) to the list of known
hosts.
Linux instance-1.us-central1-a.c.gleb-test-short-001-418811.internal 6.1.0-18-cloud-
amd64 #1 SMP PREEMPT_DYNAMIC Debian 6.1.76-1 (2024-02-01) x86_64

The programs included with the Debian GNU/Linux system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Debian GNU/Linux comes with ABSOLUTELY NO WARRANTY, to the extent
permitted by applicable law.
student@instance-1:~$
```

在 VM 內執行下列指令，安裝軟體：

```
sudo apt-get update
sudo apt-get install --yes postgresql-client
```

預期的控制台輸出內容：

```
student@instance-1:~$ sudo apt-get update
sudo apt-get install --yes postgresql-client
Get:1 https://packages.cloud.google.com/apt google-compute-engine-bullseye-stable
InRelease [5146 B]
Get:2 https://packages.cloud.google.com/apt cloud-sdk-bullseye InRelease [6406 B]
Hit:3 https://deb.debian.org/debian bullseye InRelease
Get:4 https://deb.debian.org/debian-security bullseye-security InRelease [48.4 kB]
Get:5 https://packages.cloud.google.com/apt google-compute-engine-bullseye-
stable/main amd64 Packages [1930 B]
Get:6 https://deb.debian.org/debian bullseye-updates InRelease [44.1 kB]
Get:7 https://deb.debian.org/debian bullseye-backports InRelease [49.0 kB]
...redacted...
update-alternatives: using /usr/share/postgresql/13/man/man1/psql.1.gz to provide
/usr/share/man/man1/psql.1.gz (psql.1.gz) in auto mode
Setting up postgresql-client (13+225) ...
Processing triggers for man-db (2.9.4-2) ...
Processing triggers for libc-bin (2.31-13+deb11u7) ...
```

連線至執行個體

使用 psql 從 VM 連線至主要執行個體。

在同一個 Cloud Shell 分頁中，開啟連往 instance-1 VM 的 SSH 工作階段。

使用記下的 AlloyDB 密碼 (PGPASSWORD) 值和 AlloyDB 叢集 ID，從 GCE VM 連線至 AlloyDB：

```
export PGPASSWORD=<Noted password>
```

注意：您的 VM 可能沒有權限查看 AlloyDB 執行個體。授予預設服務帳戶 `roles/alloydb.viewer`

```
export PROJECT_ID=$(gcloud config get-value project)
export REGION=us-central1
export ADBCLUSTER=alloydb-hybrid-search
export INSTANCE_IP=$(gcloud alloydb instances describe $ADBCLUSTER-pr --cluster=$ADBCLUSTER -
-region=$REGION --format="value(ipAddress)")
psql "host=$INSTANCE_IP user=postgres sslmode=require"
```

預期的控制台輸出內容：

```
student@instance-1:~$ export PGPASSWORD=CQhOi5OygD4ps6ty
student@instance-1:~$ ADBCLUSTER=alloydb-aip-01
student@instance-1:~$ REGION=us-central1
student@instance-1:~$ INSTANCE_IP=$(gcloud alloydb instances describe $ADBCLUSTER-pr
--cluster=$ADBCLUSTER --region=$REGION --format="value(ipAddress)")
gleb@instance-1:~$ psql "host=$INSTANCE_IP user=postgres sslmode=require"
psql (15.6 (Debian 15.6-0+deb12u1), server 15.5)
SSL connection (protocol: TLSv1.3, cipher: TLS_AES_256_GCM_SHA384, compression: off)
Type "help" for help.

postgres=>
```

關閉 psql 工作階段：

```
exit
```

6. 準備資料庫

我們需要建立資料庫、啟用 Vertex AI 整合功能、建立資料庫物件，以及匯入資料。

授予 AlloyDB 必要權限

將 Vertex AI 權限新增至 AlloyDB 服務代理。

使用頂端的「+」符號開啟另一個 Cloud Shell 分頁。



abc505ac4d41f24e.png

在新開啟的 Cloud Shell 分頁中執行下列指令：

```
PROJECT_ID=$(gcloud config get-value project)
gcloud projects add-iam-policy-binding $PROJECT_ID \
  --member="serviceAccount:service-$(gcloud projects describe $PROJECT_ID --
format="value(projectNumber)")@gcp-sa-alloydb.iam.gserviceaccount.com" \
  --role="roles/aiplatform.user"
```

預期的控制台輸出內容：

```
student@cloudshell:~ (test-project-001-402417)$ PROJECT_ID=$(gcloud config get-value
project)
Your active configuration is: [cloudshell-11039]
student@cloudshell:~ (test-project-001-402417)$ gcloud projects add-iam-policy-
binding $PROJECT_ID \
  --member="serviceAccount:service-$(gcloud projects describe $PROJECT_ID --
format="value(projectNumber)")@gcp-sa-alloydb.iam.gserviceaccount.com" \
  --role="roles/aiplatform.user"
Updated IAM policy for project [test-project-001-402417].
bindings:
- members:
  - serviceAccount:service-4470404856@gcp-sa-alloydb.iam.gserviceaccount.com
    role: roles/aiplatform.user
- members:
  ...
etag: BwYIEbe_Z3U=
version: 1
```

在分頁中執行「exit」指令，關閉分頁：

```
exit
```

建立資料庫

建立名為 quickstart 的資料庫。

在 GCE VM 工作階段中執行下列指令：

注意：如果 SSH 工作階段已終止，您需要重設環境變數，例如：

```
export PGPASSWORD=<Noted password>

export PROJECT_ID=$(gcloud config get-value project)

export REGION=us-central1

export ADBCLUSTER=alloydb-hybrid-search

export INSTANCE_IP=$(gcloud alloydb instances describe $ADBCLUSTER-pr --cluster=$ADBCLUSTER
--region=$REGION --format="value(ipAddress)")
```

建立資料庫：

```
psql "host=$INSTANCE_IP user=postgres" -c "CREATE DATABASE quickstart_db"
```

預期的控制台輸出內容：

```
student@instance-1:~$ psql "host=$INSTANCE_IP user=postgres" -c "CREATE DATABASE
quickstart_db"
CREATE DATABASE
student@instance-1:~$
```

啟用 Vertex AI 整合功能

在資料庫中啟用 Vertex AI 整合功能和 pgvector 擴充功能。

在 GCE VM 中執行：

```
psql "host=$INSTANCE_IP user=postgres dbname=quickstart_db" -c "CREATE EXTENSION IF NOT
EXISTS google_ml_integration CASCADE"
psql "host=$INSTANCE_IP user=postgres dbname=quickstart_db" -c "CREATE EXTENSION IF NOT
EXISTS vector"
```

預期的控制台輸出內容：

```
student@instance-1:~$ psql "host=$INSTANCE_IP user=postgres dbname=quickstart_db" -c
"CREATE EXTENSION IF NOT EXISTS google_ml_integration CASCADE"
psql "host=$INSTANCE_IP user=postgres dbname=quickstart_db" -c "CREATE EXTENSION IF
NOT EXISTS vector"
CREATE EXTENSION
CREATE EXTENSION
student@instance-1:~$
```

匯入資料

下載準備好的資料，然後匯入新資料庫。

在 GCE VM 中執行：

```
gcloud storage cat gs://cloud-training/gcc/gcc-tech-004/cymbal_demo_schema.sql |psql
"host=$INSTANCE_IP user=postgres dbname=quickstart_db"
gcloud storage cat gs://cloud-training/gcc/gcc-tech-004/cymbal_products.csv |psql
"host=$INSTANCE_IP user=postgres dbname=quickstart_db" -c "\copy cymbal_products from stdin
csv header"
gcloud storage cat gs://cloud-training/gcc/gcc-tech-004/cymbal_inventory.csv |psql
"host=$INSTANCE_IP user=postgres dbname=quickstart_db" -c "\copy cymbal_inventory from stdin
csv header"
gcloud storage cat gs://cloud-training/gcc/gcc-tech-004/cymbal_stores.csv |psql
"host=$INSTANCE_IP user=postgres dbname=quickstart_db" -c "\copy cymbal_stores from stdin csv
header"
```

預期的控制台輸出內容：

```

student@instance-1:~$ gcloud storage cat gs://cloud-training/gcc/gcc-tech-
004/cymbal_demo_schema.sql |psql "host=$INSTANCE_IP user=postgres
dbname=quickstart_db"
SET
SET
SET
SET
SET
  set_config
-----

(1 row)
SET
SET
SET
SET
SET
SET
CREATE TABLE
ALTER TABLE
CREATE TABLE
ALTER TABLE
CREATE TABLE
ALTER TABLE
CREATE TABLE
ALTER TABLE
CREATE SEQUENCE
ALTER TABLE
ALTER SEQUENCE
ALTER TABLE
ALTER TABLE
ALTER TABLE
student@instance-1:~$ gcloud storage cat gs://cloud-training/gcc/gcc-tech-
004/cymbal_products.csv |psql "host=$INSTANCE_IP user=postgres dbname=quickstart_db"
-c "\copy cymbal_products from stdin csv header"
COPY 941
student@instance-1:~$ gcloud storage cat gs://cloud-training/gcc/gcc-tech-
004/cymbal_inventory.csv |psql "host=$INSTANCE_IP user=postgres dbname=quickstart_db"
-c "\copy cymbal_inventory from stdin csv header"
COPY 263861
student@instance-1:~$ gcloud storage cat gs://cloud-training/gcc/gcc-tech-
004/cymbal_stores.csv |psql "host=$INSTANCE_IP user=postgres dbname=quickstart_db" -c
"\copy cymbal_stores from stdin csv header"
COPY 4654
student@instance-1:~$

```

7. 生成向量嵌入

匯入資料後，我們會有以下資料表：`cymbal_products` 儲存產品資訊、`cymbal_inventory` 追蹤各商店的商品庫存，以及 `cymbal_stores` 商店清單。如要對產品執行語意搜尋，我們需要使用 `initialize_embeddings` 函式生成產品說明的向量嵌入。我們會使用 Vertex AI 整合功能，根據產品說明計算向量資料，並將資料新增至表格。如要進一步瞭解使用的技術，請參閱[說明文件](#)。

如要使用整合功能，請從 VM 使用 AlloyDB 執行個體 IP 和 postgres 密碼，透過 psql 連線至資料庫：

```
psql "host=$INSTANCE_IP user=postgres dbname=quickstart_db"
```

確認 `google_ml_integration` 擴充功能版本。

```
SELECT extversion FROM pg_extension WHERE extname = 'google_ml_integration';
```

版本必須為 1.5.2 以上。以下是輸出內容範例：

```
quickstart_db=> SELECT extversion FROM pg_extension WHERE extname =  
'google_ml_integration';  
  extversion  
-----  
  1.5.2  
(1 row)
```

預設版本應為 1.5.2 以上，但如果執行個體顯示較舊的版本，可能需要更新。檢查執行個體是否已停用維護作業。

注意：後端會自動管理擴充功能版本。您可以在 psql 工作階段中執行下列指令，嘗試更新擴充功能：

```
ALTER EXTENSION google_ml_integration UPDATE;
```

如果失敗，您也可以呼叫程序，為套件啟用預覽版本 (請勿在正式版中使用預覽版本)

```
CALL google_ml.upgrade_to_preview_version();
```

我們會使用批次嵌入生成功能來提高效率。如要進一步瞭解不同的嵌入生成選項和技術，請參閱[指南](#)。如要使用批次嵌入功能，我們必須啟用 `google_ml_integration.enable_faster_embedding_generation`

```
show google_ml_integration.enable_faster_embedding_generation;
```

如果旗標位置正確，預期輸出內容如下：

```
quickstart_db=> show google_ml_integration.enable_faster_embedding_generation;  
  google_ml_integration.enable_faster_embedding_generation  
-----  
  on  
(1 row)
```

但如果顯示「off」，則需要更新執行個體。如[說明文件](#)所述，您可以使用網頁版控制台或 gcloud 指令執行這項操作。以下說明如何使用 gcloud 指令執行這項操作：

```
export PROJECT_ID=$(gcloud config get-value project)
export REGION=us-central1
export ADBCLUSTER=alloydb-hybrid-search
gcloud beta alloydb instances update $ADBCLUSTER-pr \
  --database-flags google_ml_integration.enable_faster_embedding_generation=on \
  --region=$REGION \
  --cluster=$ADBCLUSTER \
  --project=$PROJECT_ID \
  --update-mode=FORCE_APPLY
```

這可能需要幾分鐘的時間，但最終標記值應會切換為「開啟」。完成後即可繼續進行後續步驟。

注意：如果 SSH 工作階段已終止，您需要重設環境變數，例如：

```
export PGPASSWORD=<Noted password>
export INSTANCE_IP=<AlloyDB Instance IP>
```

```
psql "host=$INSTANCE_IP user=postgres dbname=quickstart_db"
```

在連線至資料庫的 psql 工作階段中，建立新資料欄，在 `cymbal_products` 中儲存嵌入內容

```
ALTER TABLE cymbal_products ADD COLUMN product_embedding vector(768);
```

預期的控制台輸出內容：

```
quickstart_db=> ALTER TABLE cymbal_products ADD COLUMN product_embedding vector(768);
ALTER TABLE
quickstart_db=>
```

最後，我們也希望在函式呼叫中加入 `incremental_refresh_mode` 引數，以便在資料欄值變更時重新整理嵌入內容。這會增加資料庫的負擔，但為了自動讓嵌入項目與內容保持同步，我們必須做出取捨。如要手動更新嵌入內容，請參閱[說明文件](#)中的操作說明。

現在將所有內容放在一起並產生嵌入，我們使用 `initialize_embeddings` 函式，並傳遞 `batch_size` (50) 做為批次提示，並將 `incremental_refresh_mode` 設為 `transactional`

```
CALL ai.initialize_embeddings(
  model_id => 'text-embedding-005',
  table_name => 'cymbal_products',
  content_column => 'product_description',
  embedding_column => 'product_embedding',
  batch_size => 50,
  incremental_refresh_mode => 'transactional'
);
```

現在，如果我們在資料表中插入 `product_embedding` 資料欄的 `NULL` 值，

```
INSERT INTO "cymbal_products" ("uniq_id", "crawl_timestamp", "product_url", "product_name",
"product_description", "list_price", "sale_price", "brand", "item_number", "gtin",
"package_size", "category", "postal_code", "available", "product_embedding") VALUES
('fd604542e04b470f9e6348e640cff794', NOW(), 'https://example.com/new_product', 'New Cymbal
Product', 'This is a new cymbal product description.', 199.99, 149.99, 'Example Brand',
'EB123', '1234567890', 'Single', 'Cymbals', '12345', TRUE, NULL);
```

現在查詢剛插入的資料列時，會發現 `product_embedding` 欄已自動更新。

```
SELECT uniq_id, (product_embedding::real[])[1:5] as product_embedding FROM cymbal_products
WHERE uniq_id='fd604542e04b470f9e6348e640cff794';
```

輸出內容應如下所示：

```
quickstart_db=> SELECT uniq_id,(product_embedding::real[])[1:5] as product_embedding
FROM cymbal_products WHERE uniq_id='fd604542e04b470f9e6348e640cff794';
```

uniq_id	product_embedding
fd604542e04b470f9e6348e640cff794	{0.015003494,-0.005349732,-0.059790313,-0.0087091,-0.0271452}

(1 row)

Time: 3.295 ms

步驟 8 · 約 10 分鐘

8. 建立向量索引

為提升向量搜尋效能，我們將新增 [ScaNN 索引](#)。

建立 ScaNN 索引

如要建構 SCANN 索引，我們還需要啟用一個擴充功能。這個擴充功能 `alloydb_scann` 提供介面，可使用 Google 的 ScaNN 演算法處理 ANN 類型的向量索引。

```
CREATE EXTENSION IF NOT EXISTS alloydb_scann;
```

預期輸出內容：

```
quickstart_db=> CREATE EXTENSION IF NOT EXISTS alloydb_scann;
CREATE EXTENSION
Time: 27.468 ms
quickstart_db=>
```

索引可以在 MANUAL 或 AUTO 模式下建立。系統預設會啟用「MANUAL」模式，您可以建立及維護索引，就像其他索引一樣。但如果啟用「自動」模式，您就能建立索引，不需要自行維護。如要詳細瞭解所有選項，請參閱[說明文件](#)。在本例中，我們沒有足夠的資料列，無法在 AUTO 模式下建立索引，因此我們將以 MANUAL 模式建立索引，並納入微調參數。如要瞭解如何調整索引參數，請參閱[說明文件](#)。

我們必須啟用 `scann.enable_preview_features` 旗標，才能修改微調參數。在 Cloud Shell 中

```
export PROJECT_ID=$(gcloud config get-value project)
export REGION=us-central1
export ADBCLUSTER=alloydb-hybrid-search
gcloud beta alloydb instances update $ADBCLUSTER-pr \
  --database-flags scann.enable_preview_features=on \
  --region=$REGION \
  --cluster=$ADBCLUSTER \
  --project=$PROJECT_ID \
  --update-mode=FORCE_APPLY
```

這可能需要幾分鐘的時間，但最終標記值應會切換為「開啟」。設定標記後，我們可以切換回 VM 上的 `psql` 工作階段，並使用調整參數建立索引。

```
CREATE INDEX cymbal_products_embeddings_scann ON cymbal_products
  USING scann (product_embedding cosine)
  WITH (mode='MANUAL', num_leaves=31, max_num_levels = 2);
```

預期輸出內容：

```
quickstart_db=> CREATE INDEX cymbal_products_embeddings_scann ON cymbal_products
  USING scann (product_embedding cosine)
  WITH (num_leaves=31, max_num_levels = 2);
CREATE INDEX
quickstart_db=>
```

檢查索引使用情形

現在我們可以在 *EXPLAIN* 模式下執行向量搜尋查詢，並驗證是否正在使用索引。

```
EXPLAIN (analyze)
WITH trees as (
SELECT
    cp.product_name,
    left(cp.product_description,80) as description,
    cp.sale_price,
    cs.zip_code,
    cp.uniq_id as product_id
FROM
    cymbal_products cp
JOIN cymbal_inventory ci on
    ci.uniq_id=cp.uniq_id
JOIN cymbal_stores cs on
    cs.store_id=ci.store_id
    AND ci.inventory>0
    AND cs.store_id = 1583
ORDER BY
    (cp.product_embedding <=> embedding('text-embedding-005','What kind of fruit trees
grow well here?'))::vector) ASC
LIMIT 1)
SELECT json_agg(trees) FROM trees;
```

預期輸出內容 (為求明確，已遮蓋部分內容)：


```
...
Aggregate (cost=16.59..16.60 rows=1 width=32) (actual time=2.875..2.877 rows=1
loops=1)
-> Subquery Scan on trees (cost=8.42..16.59 rows=1 width=142) (actual
time=2.860..2.862 rows=1 loops=1)
-> Limit (cost=8.42..16.58 rows=1 width=158) (actual time=2.855..2.856 rows=1
loops=1)
-> Nested Loop (cost=8.42..6489.19 rows=794 width=158) (actual time=2.854..2.855
rows=1 loops=1)
-> Nested Loop (cost=8.13..6466.99 rows=794 width=938) (actual time=2.742..2.743
rows=1 loops=1)
-> Index Scan using cymbal_products_embeddings_scann on cymbal_products cp
(cost=7.71..111.99 rows=876 width=934) (actual time=2.724..2.724 rows=1 loops=1)
Order By: (embedding <=>
'[0.008864171,0.03693164,-0.024245683,-0.00355923,0.0055611245,0.015985578,...
<redacted>...5685,-0.03914233,-0.018452475,0.00826032,-0.07372604]':::vector)
...
```

從輸出內容中，我們清楚看到查詢使用「Index Scan using cymbal_products_embeddings_scann on cymbal_products」。

9. 全文搜尋索引

AlloyDB 支援原生 PostgreSQL 支援的所有[索引類型](#)，可進行全文搜尋。選擇索引時，請考量搜尋速度、索引建構時間、更新速度，以及所需特定搜尋功能 (例如詞組搜尋或關聯性排名) 之間的平衡。

在我們的範例中，我們會使用 RUM 擴充功能，執行效能更高的全文搜尋作業。RUM 會直接在索引中儲存位置資訊，藉此改善標準 GIN 索引，讓您不必存取資料表資料，就能執行更快速的詞組搜尋和關聯性排名。

您可以使用 AlloyDB Studio 或繼續使用 `psql` 用戶端啟用 rum 擴充功能

建立 RUM 索引

```
CREATE EXTENSION IF NOT EXISTS rum;
```

如要在 `cymbal_products` 資料表中搜尋產品說明，我們需要建立一個欄，將產品說明儲存為 `tsvector`。這個資料欄會自動儲存處理過的文字，並提升查詢效能。

```
ALTER TABLE cymbal_products
ADD COLUMN product_search_vector tsvector
GENERATED ALWAYS AS (to_tsvector('english', product_description)) STORED;
```

現在，我們可以為 `product_search_vector` 資料欄建立新的 RUM 索引。

```
CREATE INDEX cymbal_products_rum
ON cymbal_products
USING rum (product_search_vector rum_tsvector_ops);
```

如要使用索引查詢資料表，請執行下列查詢，搜尋「cherry tree」的相符項目。 `<=>` 運算子會直接從索引計算文件與查詢之間的相關分數或距離。

```
SELECT product_name, product_description
FROM cymbal_products
WHERE product_search_vector @@ to_tsquery('english', 'cherry <-> tree')
ORDER BY product_search_vector <=> to_tsquery('english', 'cherry <-> tree');
```

10. 執行混合型搜尋

`google_vector_utils.hybrid_search()` 函式可讓您合併多種搜尋類型的結果，例如向量搜尋和全文搜尋。這項函式會使用倒數排名融合 (RRF) 演算法，將各搜尋元件的排名結果融合成單一整合清單。與單一搜尋類型相比，這種做法可提供更相關的結果。

`hybrid_search()` 函式會動態建構及執行單一 SQL 查詢。並為您定義的每個搜尋元件建立一般資料表運算式 (CTE)。接著，函式會合併所有 CTE 的結果，並計算每份文件的最終 RRF 分數，產生統一的排名清單。

如要使用這項功能，我們必須在主要執行個體中開啟 `enable_preview_ai_functions`。在 Cloud Shell 中執行下列指令：

```
export PROJECT_ID=$(gcloud config get-value project)
export REGION=us-central1
export ADBCLUSTER=alloydb-hybrid-search
gcloud beta alloydb instances update $ADBCLUSTER-pr \
  --database-flags google_ml_integration.enable_preview_ai_functions=on \
  --region=$REGION \
  --cluster=$ADBCLUSTER \
  --project=$PROJECT_ID \
  --update-mode=FORCE_APPLY
```

下列查詢會將先前的向量搜尋問題與全文搜尋問題合併。這是非常簡單的混合型搜尋查詢，您可以嘗試更複雜的查詢，例如在向量搜尋元件中使用「比房子高的樹」，並在 FTS 元件中使用「加州」。

```
SELECT score, id, p.product_name
FROM ai.hybrid_search(
  search_inputs => ARRAY[
    '{
      "data_type": "vector",
      "table_name": "cymbal_products",
      "key_column": "uniq_id",
      "vec_column": "product_embedding",
      "distance_operator": "public.<=>",
      "limit": 5,
      "query_vector": "ai.embedding('text-embedding-005', 'cherry')::vector"
    }'::JSONB,
    '{
      "data_type": "text",
      "table_name": "cymbal_products",
      "key_column": "uniq_id",
      "text_column": "product_search_vector",
      "limit": 5,
      "ranking_function": "<=>",
      "query_text_input": "tree"
    }'::JSONB
  ]
) JOIN cymbal_products p ON id = p.uniq_id;
```

預期的輸出內容：

```
"score","id","product_name"
"0.00819672631147241","d536e9e823296a2eba198e52dd23e712","Cherry Tree"
"0.015873015873015872","23e41a71d63d8bbc9bdfa1d118cfddc5","Apple Tree"
"0.00819672631147241","dc789a2f87b142e94e6e325689482af9","Oak Tree"
"0.008064521129029258","f5c70d62ccf3118d73863bf3b17edcbe","Cypress Tree"
"0.008064521129029258","b70c44b1a38c0a2329fa583c9109a80f","Peach Tree"
```

結果中會顯示 `id`，這是 `key_column` 指定的值，而 `score` 則是 RRF 計算出的最終值。倒數排名融合 (RRF) 是一種以排名為準的演算法，可為每份文件指派分數，將多份搜尋結果排名清單合併為一份。這項分數是根據 RRF 在所有貢獻清單中的倒數排名計算，排名較高的文件貢獻度較高。使用參數中的 `include_json_output => true`，系統會傳回 `detail_json` 資料欄，其中包含每個元件的分數計算明細。

全文搜尋最適合尋找特定字詞或完全相符的內容，向量搜尋則擅長尋找同義字和意圖，即使字詞不相符也沒問題。混合搜尋結合這兩種方法，確保使用者獲得的結果不僅在字面上準確，在語意上也很相關

11. 清理環境

完成實驗室後，請銷毀 AlloyDB 執行個體和叢集。

刪除 AlloyDB 叢集和所有執行個體

如果您使用過 AlloyDB 試用版，如果您打算使用試用叢集測試其他實驗室和資源，請勿刪除試用叢集。您無法在同一個專案中建立其他試用叢集。

使用 force 選項終止叢集，這也會刪除叢集中的所有執行個體。

如果連線中斷，且所有先前的設定都遺失，請在 Cloud Shell 中定義專案和環境變數：

```
gcloud config set project <your project id>
```

```
export REGION=us-central1
export ADBCLUSTER=alloydb-hybrid-search
export PROJECT_ID=$(gcloud config get-value project)
```

刪除叢集：

```
gcloud alloydb clusters delete $ADBCLUSTER --region=$REGION --force
```

注意：這個指令需要 3 到 5 分鐘才能執行完畢

預期的控制台輸出內容：

```
student@cloudshell:~ (test-project-001-402417)$ gcloud alloydb clusters delete
$ADBCLUSTER --region=$REGION --force
All of the cluster data will be lost when the cluster is deleted.

Do you want to continue (Y/n)? Y

Operation ID: operation-1697820178429-6082890a0b570-4a72f7e4-4c5df36f
Deleting cluster...done.
```

刪除 AlloyDB 備份

刪除叢集的所有 AlloyDB 備份：

注意：這個指令會毀損環境變數中指定名稱的叢集的所有資料備份

```
for i in $(gcloud alloydb backups list --filter="CLUSTER_NAME:
projects/$PROJECT_ID/locations/$REGION/clusters/$ADBCLUSTER" --format="value(name)" --sort-
by=~createTime) ; do gcloud alloydb backups delete $(basename $i) --region $REGION --quiet;
done
```

預期的控制台輸出內容：

```
student@cloudshell:~ (test-project-001-402417)$ for i in $(gcloud alloydb backups
list --filter="CLUSTER_NAME:
projects/$PROJECT_ID/locations/$REGION/clusters/$ADBCLUSTER" --format="value(name)" -
-sort-by=~createTime) ; do gcloud alloydb backups delete $(basename $i) --region
$REGION --quiet; done
Operation ID: operation-1697826266108-60829fb7b5258-7f99dc0b-99f3c35f
Deleting backup...done.
```

現在可以刪除 VM 了

刪除 GCE VM

在 Cloud Shell 中執行下列指令：

```
export GCEVM=instance-1
export ZONE=us-central1-a
gcloud compute instances delete $GCEVM \
    --zone=$ZONE \
    --quiet
```

預期的控制台輸出內容：

```
student@cloudshell:~ (test-project-001-402417)$ export GCEVM=instance-1
export ZONE=us-central1-a
gcloud compute instances delete $GCEVM \
    --zone=$ZONE \
    --quiet
Deleted
```

12. 恭喜

恭喜您完成本程式碼研究室。

涵蓋內容

- 如何部署 [AlloyDB](#) 叢集和主要執行個體
 - 如何從 [Google Compute Engine](#) VM 連線至 AlloyDB
 - 如何建立資料庫並啟用 [AlloyDB AI](#)
 - 如何將資料載入資料庫
 - 如何使用 [AlloyDB Studio](#)
 - 使用 [Vertex AI](#) 生成嵌入
 - 如何建立 [ScaNN 向量索引](#)，提升向量搜尋效果
 - 如何啟用及使用 RUM 擴充功能進行全文搜尋
 - 結合全文搜尋、向量搜尋和[倒數排名融合 \(RRF\)](#) 執行混合型搜尋
-